ChromaDB Knowledge Base

Synthetic Dataset for RAG & Embedding Projects

Generated: 2026-06-08 | Records: 500 | Format: Multi-domain

1. Product Catalogue

The following products form the core offering portfolio. Each entry contains structured metadata suitable for dense retrieval and semantic similarity search.

Neural Processor X1

SKU	NPXS-001
Category	AI/ML
Pricing	$4,999/mo

High-throughput neural inference engine supporting ONNX, TensorRT, and custom CUDA kernels. Designed for sub-10ms latency on transformer models up to 70B parameters. Includes built-in quantization (INT8/FP16) and dynamic batching. Throughput: 12,000 tokens/sec on A100 GPU.

QuantumDB 3.0

SKU	QDB-030
Category	Database
Pricing	$1,200/mo

Distributed columnar database with native vector index support (HNSW + IVF). Handles mixed workloads of OLAP queries and ANN search with ACID guarantees. Auto-sharding across up to 256 nodes. Benchmark: 95ms p99 for 1M vector queries.

CloudSync Pro

SKU	CSP-004
Category	Cloud
Pricing	$299/mo

Real-time bidirectional data sync across multi-cloud environments (AWS, GCP, Azure). Change data capture (CDC) with guaranteed exactly-once delivery. Supports Kafka, Kinesis, and Pub/Sub as streaming backends.

DataVault Enterprise

SKU	DVE-007
Category	Security
Pricing	$8,500/mo

Zero-knowledge encrypted data lake with attribute-based access control (ABAC). FIPS 140-2 Level 3 certified. Supports tokenisation, format-preserving encryption, and differential privacy for analytics over sensitive datasets.

StreamLine API

SKU	SLA-012
Category	API
Pricing	$0.002/req

Low-latency REST/gRPC API gateway with automatic rate limiting, circuit breaking, and semantic caching. Reduces LLM API costs by up to 60% via semantic deduplication of near-identical prompts using cosine similarity thresholding.

VectorStore Plus

SKU	VSP-019
Category	AI/ML
Pricing	$799/mo

Managed vector database optimised for embedding storage and retrieval. Supports 1536-dim OpenAI embeddings, 768-dim BERT, and custom dimension sizes. Metadata filtering with BM25 hybrid search. SLA: 99.99% uptime.

RAG Framework

SKU	RAGF-022
Category	DevTools
Pricing	$499/mo

End-to-end retrieval-augmented generation framework. Includes document loaders (PDF, DOCX, HTML, CSV), text splitters, embedding pipelines, and LLM chaining. Compatible with LangChain and LlamaIndex. Ships with evaluation harness (RAGAS).

LLM Gateway

SKU	LLMG-031
Category	Infrastructure
Pricing	$1,800/mo

Unified API gateway for routing requests across OpenAI, Anthropic, Cohere, and self-hosted models. Features cost tracking per team, prompt logging, PII redaction, and A/B testing for model evaluation. Latency overhead: <2ms per request.

2. Technical Documentation

2.1 Embedding Pipeline Architecture

The embedding pipeline follows a modular design comprising four stages: ingestion, preprocessing, vectorisation, and indexing. During ingestion, raw documents are parsed from supported formats (PDF, DOCX, HTML, Markdown, CSV, JSON). The preprocessing stage performs language detection, encoding normalisation (UTF-8), and HTML stripping. Chunking strategies include fixed-size (default 512 tokens, 64-token overlap), sentence-aware splitting using spaCy, and semantic chunking via cosine similarity thresholds between consecutive sentences. Each chunk is assigned a unique SHA-256 hash to enable deduplication during incremental updates.

2.2 Vector Index Configuration

ChromaDB supports two primary indexing algorithms: HNSW (Hierarchical Navigable Small World graphs) for approximate nearest-neighbour search, and flat (brute-force exact) search for small collections. HNSW parameters include M (number of bi-directional links, default 16), ef_construction (size of the dynamic candidate list during build, default 100), and ef_search (query-time parameter controlling recall vs speed tradeoff). Recommended settings for production RAG: M=32, ef_construction=200, ef_search=128. Distance metrics: L2 (Euclidean), cosine similarity, and inner product.

2.3 Metadata Filtering

ChromaDB's where clause supports rich metadata filtering using $eq, $ne, $gt, $gte, $lt, $lte, $in, $nin operators. Compound filters use $and and $or boolean operators. Example query: {'$and': [{'category': {'$eq': 'AI/ML'}}, {'score': {'$gte': 7.5}}]}. Metadata fields are stored as a JSON blob alongside the vector; filtering occurs post-ANN retrieval on the candidate set, so highly selective filters may reduce effective recall. Pre-filtering with dedicated metadata indexes is recommended for high-cardinality fields.

2.4 Retrieval Strategies

Three retrieval modes are supported. Dense retrieval uses pure vector similarity and works best for semantic paraphrase matching. Sparse retrieval (BM25) excels at keyword matching and exact term lookup. Hybrid retrieval combines both using reciprocal rank fusion (RRF) with a tunable alpha parameter (0=sparse, 1=dense). For most RAG applications alpha=0.7 yields the best balance. Re-ranking with cross-encoder models (e.g. ms-marco-MiniLM-L-12-v2) applied to top-k=50 candidates and re-ranked to top-5 consistently improves MRR@10 by 8-15%.

3. Code Examples

3.1 Initialise ChromaDB Collection

```
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.PersistentClient(path="./chroma_db")

openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="YOUR_API_KEY",
    model_name="text-embedding-3-small"
)

collection = client.get_or_create_collection(
    name="product_knowledge",
    embedding_function=openai_ef,
    metadata={"hnsw:space": "cosine", "hnsw:M": 32}
)
```

3.2 Batch Ingest from CSV

```
import pandas as pd, uuid

df = pd.read_csv("chroma_dataset.csv")

# Build documents from embedding_hint + review + description
df["document"] = (df["product"] + ". " + df["description"] + " " + df["review"])

BATCH = 100
for i in range(0, len(df), BATCH):
    chunk = df.iloc[i:i+BATCH]
    collection.add(
        ids=[f"doc-{j}" for j in chunk["id"]],
        documents=chunk["document"].tolist(),
        metadatas=chunk[["category","status","country","score","nps_score"]].to_dict("records")
    )
print(f"Indexed {len(df)} documents")
```

3.3 Semantic Query with Metadata Filter

```
results = collection.query(
    query_texts=["fast vector database for machine learning"],
    n_results=5,
    where={"$and": [
        {"category": {"$eq": "AI/ML"}},
        {"score":    {"$gte": 7.0}}
    ]},
    include=["documents", "metadatas", "distances"]
)
```

```
for doc, meta, dist in zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0]
):
    print(f"[{dist:.4f}] {meta['category']} | {doc[:80]}...")
```

3.4 Hybrid Search (Dense + BM25)

```
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunction
from rank_bm25 import BM25Okapi

# Dense retrieval
ef = SentenceTransformerEmbeddingFunction("all-MiniLM-L6-v2")
dense_results = collection.query(query_texts=[query], n_results=50)

# Sparse (BM25) retrieval
corpus = [d.split() for d in all_documents]
bm25   = BM25Okapi(corpus)
sparse_scores = bm25.get_scores(query.split())

# Reciprocal Rank Fusion
def rrf(rank, k=60): return 1.0 / (k + rank)
alpha = 0.7  # 0=sparse, 1=dense

# Combine and re-rank ...
```

4. User Feedback & Performance Metrics

The following table summarises aggregated feedback from enterprise customers across product lines. NPS scores, support ticket volumes, and churn probabilities are derived from the accompanying CSV dataset.

Product	Avg NPS	Avg Score	Churn %	Reviews	Status
Neural Processor X1	62	8.7	4.2%	312	active
QuantumDB 3.0	58	8.2	6.1%	287	active
CloudSync Pro	44	7.5	9.3%	198	active
DataVault Enterprise	71	9.1	3.8%	145	enterprise
StreamLine API	39	6.9	12.5%	423	active
VectorStore Plus	55	8.0	5.7%	256	active
RAG Framework	48	7.3	8.9%	334	trial
LLM Gateway	67	8.8	4.4%	189	enterprise

4.1 Selected Customer Testimonials

"We ingested 2TB of sensitive healthcare records with zero compliance incidents. The ABAC model let us give analysts query access without exposing PII. Remarkable product."
— Arjun P., DataVault Enterprise

"Prototyping a RAG pipeline that used to take a week now takes an afternoon. The semantic chunker alone saved us dozens of hours of prompt engineering."
— Laura M., RAG Framework

"Routing between GPT-4o and Claude depending on cost thresholds cut our monthly API spend by 41%. The latency overhead is genuinely negligible."
— Carlos W., LLM Gateway

"We run nightly batch embedding jobs on 500k documents. The throughput is consistent and the metadata filtering makes our hybrid search pipeline very clean."
— Yuki T., VectorStore Plus

"Latency dropped from 85ms to 7ms after switching to the INT8 quantised path. Our real-time recommendation system finally meets SLA requirements."
— Nina B., Neural Processor X1

5. Glossary of Key Terms

ANN: Approximate Nearest Neighbour – an algorithm that finds vectors close to a query vector without exhaustive search.

Chunking: The process of splitting long documents into smaller segments for embedding and retrieval.

ChromaDB: An open-source embedding database for storing, querying, and managing vector embeddings with metadata.

Cosine Similarity: A measure of similarity between two vectors based on the cosine of the angle between them (range: -1 to 1).

Dense Retrieval: Retrieval method that uses vector similarity search over learned dense embeddings.

Embedding: A fixed-size numerical representation of text (or other data) capturing semantic meaning.

HNSW: Hierarchical Navigable Small World – a graph-based ANN index offering excellent query speed/recall tradeoffs.

Hybrid Search: Combining dense (semantic) and sparse (keyword) retrieval, typically via Reciprocal Rank Fusion.

LLM: Large Language Model – a neural network trained on large text corpora capable of generation and reasoning.

Metadata Filtering: Applying structured filters (e.g. date range, category) alongside vector search to refine results.

NPS: Net Promoter Score – a customer loyalty metric ranging from -100 to +100.

RAG: Retrieval-Augmented Generation – augmenting LLM responses with relevant documents retrieved from a knowledge base.

Re-ranking: A second-stage model (usually a cross-encoder) that re-scores an initial candidate set for higher relevance.

RRF: Reciprocal Rank Fusion – a score combination method for merging ranked lists from multiple retrieval systems.

Sparse Retrieval: Retrieval using term-based representations (TF-IDF, BM25) that capture keyword overlap.

Vector Database: A database system optimised for storing and querying high-dimensional vector embeddings.